

# GTX-GPU: Latch-free Versioned Graph Transactions with Concurrent Snapshot Readers on GPUs

## Technical Report

Suyash Kumar

suyashkumar2005@gmail.com

Affiliation

Country

### Abstract

Dynamic graph structures for GPUs (GPMA, LPMA, Hornet, faim-Graph, SlabGraph, Meerkat) sustain hundreds of millions of edge updates per second, but none of them offers *multi-edge transactions* or *snapshot reads that run concurrently with updates*. General-purpose GPU software transactional memories provide atomicity, but their conflict detection is unaware of the graph and collapses under the skew of real-world workloads. This report presents GTX-GPU, a GPU adaptation of GTX, a write-optimized latch-free graph transaction system for CPUs. GTX-GPU keeps GTX’s versioned edge deltas, per-vertex delta blocks, destination-hashed delta chains, lazily resolved transaction descriptors and epoch-based group commit. It changes four things for the SIMT execution model: (i) *destination-precise* write-write conflict detection that validates and CASes on the same observed chain head; (ii) *block generations* with lazy chain sealing and a fused generation/fill reservation word, which lets a warp reserve slots cooperatively and never copies a block under writers; (iii) *wait-free epoch registration* with a single advancer lane that publishes a read frontier; and (iv) a *serializable* mode that uses Silo-style ordering (epoch fixed before read validation). Each benchmarked history is validated operation by operation by a checker for snapshot isolation and serializability. During development the checker found five protocol defects, all fixed before any measurement. On an RTX 4060 Laptop GPU, GTX-GPU is 14–23× faster than two-phase locking and a TL2 word-STW on the same storage when 90% of transactions are snapshot readers. With concurrent readers on Zipf workloads, serializable read-write mixes, large transactions or hub contention, the advantage grows to two to four orders of magnitude. GTX-GPU is 8.6–20× faster than GPMA+ on identical update streams. We also report where GTX-GPU loses: uncontended small transactions (0.47–0.76× of 2PL), write-only contention against a degree-free STM, and 2.5–20× memory overhead because version reclamation is not yet implemented.

### CCS Concepts

• **Information systems** → Graph-based database models; Database transaction processing; • **Computing methodologies** → Massively parallel algorithms.

### Keywords

Dynamic graph, GPU, transaction, snapshot isolation, serializability, multi-versioning, latch-free concurrency control

### 1 Introduction

Dynamic graphs change continuously: edges between users, accounts, devices or road segments are inserted and deleted while analytics and point queries run over the same graph. GPUs are an attractive home for such graphs. They offer an order of magnitude more memory bandwidth and thread-level parallelism than a CPU socket, and a line of GPU-resident dynamic-graph structures has pushed raw update rates into the hundreds of millions of edges per second. SlabGraph, for example, reports 501–641 MEdge/s for batched insertions on a TITAN V [3].

These structures share one gap. They apply updates *batch-at-a-time*, with no atomicity across edges and no consistent view for readers that run while a batch is applied [3, 5, 7, 11, 13, 17, 21]. Applications that must move an edge (delete  $(u, v)$ , insert  $(u, w)$ ), insert both directions of an undirected edge, or update an adjacency list together with a degree counter need *multi-edge transactions*. Analytics that run concurrently with updates need *snapshots*. On the GPU the only general answer today is a software transactional memory [12, 15, 18]. A word-based STM treats the adjacency structure as anonymous memory words, so every insert into a hub vertex conflicts on the hub’s degree word and on nearby hash slots.

On the CPU, GTX [19] addresses the same problem for main-memory graph systems. It stores each vertex’s adjacency as a block of *edge deltas* (versions with creation and invalidation timestamps). It indexes the deltas by destination into *delta chains*, which serve both as a lookup accelerator and as the granularity of write-write conflict detection. Transaction descriptors are resolved lazily by readers, and commit happens in groups against global read/write epochs. GTX reports up to 11× higher transaction throughput than LiveGraph, Teseo and Sortledton on write-heavy, temporally skewed workloads [9, 10, 19, 20].

This report asks whether that architecture can be moved to a GPU, and what must change. Four GPU properties make a direct port unsuitable:

- **Lockstep warps and hub skew.** Thirty-two lanes of a warp often target the same hub vertex. Per-lane atomics on the vertex’s allocation counter serialize at one L2 slice, and blocking locks inside a warp collective can deadlock.
- **No stop-the-world block growth.** GTX consolidates a full edge-delta block by copying it. On a GPU, thousands of writers may be appending to that block when it fills.
- **A weak, scoped memory model.** Every visibility edge (block publication, chain head, descriptor, epoch frontier) needs an explicit release/acquire argument at device scope.

A missing one does not fail loudly; it produces a rare wrong read.

- **Validation at scale.** A kernel commits millions of transactions in milliseconds. Comparing the final graph against a log, the usual check in GPU prototypes, cannot see lost updates or torn snapshots (§5.2).

#### Contributions.

- (1) **GTX-GPU design** (§3–§4). A GPU-resident multi-version graph store with transactional edge insertion and deletion, point lookups and adjacency scans, under snapshot isolation (SI) and under serializability (SER). GTX provides SI only [19]. The GPU-specific mechanisms are destination-precise validate-then-CAS conflict detection, block generations with lazy sealing, warp-cooperative reservation on a fused generation/fill word, wait-free epoch registration with a one-lane advancer, and Silo-ordered serializable validation [16].
- (2) **A correctness methodology for GPU transaction protocols** (§5). Six invariants and their memory-order requirements, plus a history checker (C1–C5) that validates *every* read, no-op write and scan against its snapshot, write-write exclusion, the final state and serialization-graph acyclicity. The checker found five protocol defects during development and rejects three deliberately broken protocols, two of which pass the final-state check.
- (3) **A matched evaluation** (§6). Same workload streams and seeds against strict two-phase locking, a TL2-style word STM [8] with and without a degree word, non-transactional CAS updates, and GPMA+ [13]. Results are medians of five repetitions with 95% bootstrap confidence intervals, and losses are reported next to wins.
- (4) **Falsifiable design hypotheses** (§6.7). Each GPU mechanism was stated with a falsification criterion before it was measured. One was falsified as a primary mechanism (cooperative reservation), one was supported (destination-precise conflicts), and one was not falsified (epoch commit groups).

## 2 Background and Related Work

### 2.1 GTX

GTX [19] is a main-memory, latch-free graph data system. Each vertex owns an *edge-delta block*, an append-only array of edge deltas. A delta records an operation on edge  $(u, v)$  together with a creation timestamp and an invalidation timestamp. A small *delta-chains index* per block hashes destinations to chains. A writer takes a lock bit in the chain’s index entry by CAS and aborts if the CAS fails, a granularity the authors describe as “a middle ground between vertex and edge locks”. Transactions obtain snapshots from global read/write epochs, following LiveGraph [20]. Commit is a hybrid group commit, and timestamps are written into deltas lazily after commit. GTX supports *snapshot isolation only*: it detects write-write conflicts but not read-write conflicts.

### 2.2 Dynamic graphs on GPUs

Table 1 summarizes the design space, following the recent survey of Lin, Zou and Su [11]. GPMA and GPMA+ keep edges in a sorted

**Table 1: Design space of GPU dynamic-graph structures and transactional systems.**

| System            | Representation             | Update sync.               | Txns / snapshots    |
|-------------------|----------------------------|----------------------------|---------------------|
| GPMA / GPMA+ LPMA | sorted PMA                 | locks / lock-free segments | none (batch)        |
| Hornet            | leveled PMA                | segment updates            | none                |
| faimGraph         | per-vertex arrays          | batched                    | none                |
| SlabGraph         | linked pages + reclaim     | warp per list              | none                |
| Meerkat           | per-vertex slab hash       | warp-coop. CAS             | none                |
| GPU STMs          | memory words               | locks / versions           | yes, graph-agnostic |
| GTX (CPU)         | delta blocks + chains      | latch-free, group commit   | SI                  |
| <b>GTX-GPU</b>    | delta generations + chains | validate-then-CAS, epochs  | <b>SI and SER</b>   |

packed memory array. GPMA+ replaces GPMA’s lock-based segment updates with lock-free, segment-oriented batch updates and reports up to 18.30× over GPMA [13, 14]. LPMA levels the PMA to avoid global reallocation and reports 6–20× over GPMA+ [21]. Hornet [5] and faimGraph [17] manage per-vertex arrays or pages with GPU-side allocators, and faimGraph also reclaims memory. SlabGraph [3] builds per-vertex SlabHash tables [2] updated by warp-cooperative lock-free CAS. Meerkat [7] extends SlabGraph with iterators for dynamic algorithms. All of these optimize the *throughput of a batch*. None provides transactions spanning several edges or readers that observe a consistent state while a batch is in flight.

### 2.3 Transactional memory on GPUs

GPU-STM [18] is a word- and lock-based STM designed around lockstep execution. PR-STM [15] resolves conflicts with priority rules. CSMV [12] uses a client-server, multi-versioned design. These systems provide atomic transactions over arbitrary words, but when applied to a graph they see only anonymous words: a degree counter or an open-addressing probe sequence becomes a conflict hot spot unrelated to the edges a transaction actually touches. Our STM baseline (§6.1) follows TL2 [8], the canonical word-STM design point: a global version clock, versioned lock records, commit-time sorted locking, and read-only transactions without read sets.

### 2.4 Isolation levels

Snapshot isolation [4] lets a transaction read a consistent snapshot and commit if no concurrent committed transaction wrote the same items (first-committer-wins). It admits write skew. Serializability forbids it, and a history is serializable iff its direct serialization graph of ww, wr and rw dependencies is acyclic [1]. Serializable SI [6] and Silo [16] add read validation on top of snapshot or optimistic execution. GTX-GPU follows Silo’s ordering: a writer’s commit epoch is fixed *before* its reads are validated.

## 3 GTX-GPU Architecture

GTX-GPU manages a directed graph with a fixed vertex set  $V$  entirely in GPU global memory. A single persistent kernel executes a stream of transactions, each a sequence of  $K$  operations drawn from  $\{\text{INSERT}(u, v), \text{DELETE}(u, v), \text{LOOKUP}(u, v), \text{SCAN}(u)\}$ . Figure 1

**Table 2: Data layout (all in GPU global memory).**

| Structure         | Contents                                                                                                        |
|-------------------|-----------------------------------------------------------------------------------------------------------------|
| Delta (32 B)      | dstop (dst:30, op:2), prev, owner (txid), cts, its (0, TAG txid, or epoch); one arena, bump-allocated per block |
| Block (32 B)      | base, cap, nch, chBase, prevGen; a <i>generation</i> ; capacity grows by 8×                                     |
| heads H[]         | idx:31   SEAL:1; nch = cap/chRatio heads per generation (default: one per slot)                                 |
| vcurl[u]          | current generation, read by readers; published <i>before</i> vfill                                              |
| vfill[u] (64 b)   | (generation«32)   slots reserved; writers' reservation word                                                     |
| desc[txid] (64 b) | 0 ACTIVE, 1 ABORTED, (e«2)   2 COMMITTED at epoch <i>e</i> ; fresh per attempt                                  |
| E (64 b)          | (open epoch«32)   writers registered; one word ⇒ wait-free registration                                         |
| done[2], frontier | completion counters (epoch parity), published read epoch                                                        |

shows the storage and the commit machinery. Table 2 lists the exact layout.

### 3.1 Versioned edge deltas

Every write creates a new *delta* (Table 2), never an in-place update. A delta records its destination and operation (INS, DEL, or VOID for a reserved slot that was never linked), the index of the previous delta on its chain, the id of the transaction that created it (owner), its creation epoch cts and its invalidation stamp its. A committed deletion followed by a re-insertion is simply a newer INS version. The state of an edge for a transaction is the operation of the newest version visible to it.

### 3.2 Block generations and delta chains

GTX consolidates a full edge-delta block into a larger one. GTX-GPU never copies a block while writers are active. When a vertex's current block fills, exactly one writer (the lane whose reservation lands on slot cap) allocates a generation eight times larger. The new generation is linked in *front* of the old one through prevGen and published by release stores to vcurl[u] and then vfill[u]. Each generation has its own chain heads, indexed by a hash of the destination; the default is one head per slot. A chain may therefore continue into older generations. To keep the order of versions of one key total, the first writer of generation *g*+1 that walks an older generation *seals* that generation's head for its destination (fetch\_or SEAL) before walking it. From then on, all installs for that destination go to the newer generation (invariant I2, §5.1).

### 3.3 Execution model

The kernel is persistent. Warp 0, lane 0 is the *advancer*, which closes epochs and publishes the read frontier. Every other lane is a worker that takes transactions from a global counter (one warp-aggregated atomicAdd per warp) and runs them to commit, retrying with randomized exponential backoff after an abort. Warps execute attempt() collectively, so lanes of one warp share reservation, commit registration and scans. A transaction gives up after maxAttempts attempts or on resource exhaustion; give-ups are counted and reported, never hidden. For each committed transaction the kernel records (*rts*, *cts*, *txid*, *attempts*) and the result of every operation. These records feed the history checker.

**Algorithm 1: Warp-cooperative slot reservation**


---

**Input:** vertex *u* for lanes that need a slot

```

1  grp ← __match_any_sync(FULL, u);
2  if lane is leader of grp then
3    | w ← vfill[u].fetch_add(popc(grp), acq_rel)
4  broadcast w; s ← low(w) + rank(lane, grp); g ← high(w);
5  if g = NONE ∧ s = 0 then
6    | allocBlock(u, NONE); // unique creator
7  else if s < cap(g) then
8    | return (g, base(g) + s)
9  else if s = cap(g) then
10   | allocBlock(u, g); // unique grower
11 else
12   | wait (__nanosleep); retry
13 allocBlock(u, old):
14   B[new] ← {base, cap « 3, nch, chBase, prevGen=old}; fence;
15   vcurl[u] ← new (release); vfill[u] ← (new « 32) | 0 (release);

```

---

## 4 Transactions and Concurrency Control

### 4.1 Visibility and operation semantics

Transaction *T* begins with snapshot *rts* ← frontier (acquire) and a fresh descriptor in state ACTIVE. A version of edge (*u*, *v*) is *visible* to *T* iff *T* created it, or its creator committed at an epoch ≤ *rts*. Visibility is decided by the creator's descriptor (read lazily) or by the stamped cts. The state of (*u*, *v*) for *T* is the operation of the newest visible version; an edge with no visible version is absent.

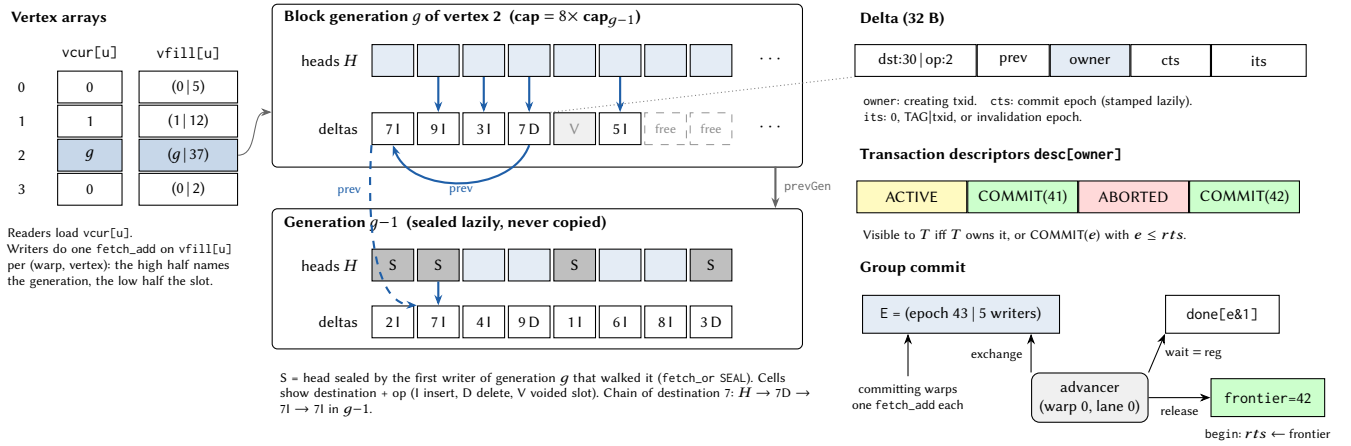
- LOOKUP(*u*, *v*) returns the state, with read-your-writes, and records which delta it observed.
- INSERT(*u*, *v*) on a present edge (and DELETE on an absent one) is a *no-op* that behaves as a read. Otherwise it appends an INS (DEL) delta. A DEL also tags the superseded INS with its := TAG|*T*.
- SCAN(*u*) returns every visible INS of *u* that is not invalidated by a version visible to *T* (degree and an order-independent hash of the neighbour set).
- ABORT publishes ABORTED. Nothing is rolled back: the aborted deltas are invisible forever.

### 4.2 Cooperative reservation

A writer needs one delta slot per effective write. Algorithm 1 obtains it. Lanes of a warp that target the same vertex are grouped with \_\_match\_any\_sync. The group leader performs one fetch\_add of the group size on vfill[*u*] and broadcasts the result, and each lane derives its slot from its rank in the group. Because vfill packs the generation in its high half, the RMW returns the generation and the slot atomically, and a lane can never pair a slot index with the wrong block. Only the unique lane that receives slot cap grows the vertex. Lanes that overshoot wait (with \_\_nanosleep) and retry. This wait is bounded by one allocBlock under independent thread scheduling. The RMW must be acq\_rel: defect D8 (§5.3) showed that a relaxed reservation leaves no happens-before edge to the grower's publication of the new generation.

### 4.3 Destination-precise conflict detection

GTX detects write-write conflicts at the granularity of a delta chain: a writer aborts if the chain's newest live delta is not visible to it,



**Figure 1: GTX-GPU storage and commit machinery.** A vertex’s adjacency is a list of block generations, newest first. Each generation has one chain head per slot, hashed by destination, and a bump-allocated array of 32-byte deltas. A writer reserves a slot with one RMW on `vfill[u]`, validates the chain of its destination and links the delta by CAS on the chain head it validated. Readers decide visibility through the creator’s descriptor and the published epoch frontier.

---

**Algorithm 2: Write (insert/delete) of edge  $(u, v)$  by  $T$** 


---

```

1  $(g, slot) \leftarrow \text{reserveSlot}(u)$ ;
2  $h_0 \leftarrow H[\text{head}(g, v)].\text{load}(\text{acquire})$ ;
3 if  $h_0.\text{SEAL}$  then  $\text{void}(slot)$ ; retry reservation;
4  $r \leftarrow \text{walkGens}(g, h_0, v, \text{seal}=\text{true})$ ; // newest version of  $v$ 
5 foreach delta  $d$  of  $v$  met newest-first do
6   if  $d.\text{owner} = T$  then found;
7   else if  $d.\text{aborted}$  then skip;
8   else if  $d.\text{active}$  then  $\text{abort}(\text{WW\_PENDING})$ ;
9   else if  $d.\text{committed} > rts$  then  $\text{abort}(\text{WW\_LATE})$ ;
10  else found;
11 decide effective / no-op from  $r$ ; on no-op or abort:  $\text{void}(slot)$ ;
12 write delta  $\{dstop, prev=h_0, owner=T\}$ ;
13 while  $\neg \text{CAS}(H[\text{head}(g, v)], h_0 \rightarrow slot) (\text{acq\_rel})$  do
14    $h_1 \leftarrow \text{observed head}$ ;
15   if  $h_1.\text{SEAL}$  then  $\text{void}(slot)$ ; retry reservation;
16   if a non-aborted foreign version of  $v$  lies in  $h_1..h_0$  then  $\text{void}(slot)$ ; abort;
17    $prev \leftarrow h_1$ ;  $h_0 \leftarrow h_1$ ;
18 if  $op = \text{DEL}$  then  $\text{its}(\text{superseded}) \leftarrow \text{TAG}|T$ ;

```

---

whichever destination that delta belongs to. With many destinations hashed onto few chains and large transactions, most of these aborts are *false*. GTX-GPU checks only versions of the same destination (Algorithm 2), and it guarantees exclusion by validating and installing against the *same* observed head  $h_0$ . If the CAS fails, only the new prefix  $h_1..h_0$  is walked, and any non-aborted foreign version of  $v$  in it aborts the writer. The walk applies first-updater-wins: an ACTIVE foreign version aborts with WW\_PENDING, and a version committed after  $rts$  aborts with WW\_LATE. The GTX-faithful rule remains available ( $\text{dstconf}=0$ ) for ablation and reports its extra aborts as AB\_CHAIN\_FALSE.

#### 4.4 Group commit with epochs

GTX’s group commit uses global read and write epochs. On the GPU, the obvious registration protocol (read E, increment an in-flight counter, re-read E, retry on change) *livelocked* for seconds, because

---

**Algorithm 3: Commit (per warp) and the advancer**


---

```

1  $W \leftarrow \text{lanes alive with } \geq 1 \text{ effective write}$ ;
2  $e \leftarrow E.\text{fetch\_add}(\text{popc}(W), \text{acq\_rel}) \gg 32$ ; // leader; before validation
3 if SER then
4   validate point reads: newest non-aborted foreign version of  $(u, v)$  is the
   observed one;
5   validate scans: every foreign non-aborted delta of  $u$  visible at  $rts$  and
   not invalidated;
6 fence;  $\text{desc}[T] \leftarrow \text{alive} ? \text{COMMITTED}(e) : \text{ABORTED}$ ;  $\_\_\text{syncwarp}$ ;
7 leader: fence;  $\text{done}[e \& 1] += \text{popc}(W)$ ;
8 stamp:  $\text{cts}(\text{own deltas}) \leftarrow e$ ;  $\text{its}(\text{superseded}) \leftarrow e$ ;
9 advancer (one lane);
10 while true do
11    $e \leftarrow E.\text{hi}$ ;  $\text{reg} \leftarrow E.\text{exchange}((e+1) \ll 32).\text{lo}$ ;
12   wait  $\text{done}[e \& 1] = \text{reg}$ ;  $\text{done}[e \& 1] \leftarrow 0$ ;  $\text{frontier} \leftarrow e$  (release);

```

---

the advancer bumps the epoch about once per microsecond (defect D5). GTX-GPU makes registration a single RMW (Algorithm 3). E packs the open epoch with the number of writers registered in it, so a warp’s leader registers all its committing lanes with one `fetch_add` and receives the epoch in the same instruction. The advancer closes an epoch with one exchange, which atomically returns the exact number of registered writers, waits until `done[e&1]` reaches that count, and only then publishes `frontier := e` with a release store. Readers take snapshots with one acquire load. After commit, lanes lazily stamp `cts` on their deltas and `its` on superseded versions, so later readers skip the descriptor lookup.

A transaction that performed no effective write commits at  $\text{cts} = rts$  without registering. It publishes ABORTED rather than COMMITTED because it may own reserved-then-voided slots, which must never become “committed in the past” (defect D7).

#### 4.5 Serializable mode

SI permits write skew. GTX-GPU’s SER mode adds optimistic read validation at commit. For a point read or a no-op write, the newest

**Table 3: Cost per effective write (uncontended).**

| Step        | Dependent global round trips                                      | Atomics                     |
|-------------|-------------------------------------------------------------------|-----------------------------|
| reservation | 1 RMW on <code>vfill[u]</code> (+1 load of $B[g]$ )               | 1 per (warp, vertex)        |
| walk        | 1 + chain length ( $\approx 1$ ) + 1 per older gen.               | 1 seal per (old gen, chain) |
| install     | 1 CAS                                                             | 1                           |
| commit      | 1 RMW on $E$ , 1 on done per warp; per lane 1 desc store + stamps | 2 per warp                  |

non-aborted foreign version of  $(u, v)$  must still be the one observed. For a scan, every foreign, non-aborted delta of  $u$  must be visible at  $rts$  and not invalidated by a non-aborted foreign transaction; this is a predicate (phantom) check. The ordering matters. In our first design the epoch was taken *after* validation, so an rw anti-dependency  $T \rightarrow U$  could run backwards in commit order, and read-only transactions reading from the frontier saw non-serializable states (defect D6, 5 histories with up to 40,535 serialization-graph back edges). Fixing the epoch *before* validation (Silo order [16]) makes every ww, wr and rw dependency non-decreasing in commit epoch. A frontier snapshot is then a dependency-closed prefix, and read-only transactions serialize at  $rts$  with no validation at all.

#### 4.6 Reads and scans

LOOKUP walks the chain(s) of  $v$  from `vcursor[u]` newest-first and returns the first version visible to  $T$ . SCAN visits every slot  $[0, \min(fill, cap))$  of every generation. By default the whole warp strides one vertex (warp-cooperative scan), which is essential for power-law graphs (§6.4). Readers never write shared state, never wait, and never abort in SI mode.

#### 4.7 Cost model

Table 3 gives the uncontended cost of one effective write. The dominant fixed cost compared with a lock-based hash table is versioning: a 32-byte delta, a chain head and descriptor/epoch bookkeeping per attempt. Scans cost  $\Theta(\text{slots in all generations})$ , including aborted and voided slots until reclamation, which is specified but not implemented (§7).

### 5 Correctness

#### 5.1 Invariants

- I1 Single writer per chain position.** For every key, non-aborted versions are totally ordered by chain position, and that order equals commit-epoch order. This follows from three rules: an install requires the newest non-aborted version to be  $T$ 's own or committed  $\leq rts$ ; the CAS is on the same observed head; and a failed CAS triggers a prefix re-walk.
- I2 Generation monotonicity.** If a version of  $v$  exists in generation  $g+1$ , the head of  $v$ 's chain in every older generation is sealed, so every install into  $g$  precedes every install into  $g+1$  for that key.
- I3 Frontier safety.**  $\text{frontier} \geq e$  implies that every writer registered in  $e$  has published its final descriptor. Registration and closing are RMWs on one word, and the advancer waits for the exact count.

**I4 No torn visibility.** A transaction's versions become visible to a snapshot all-or-nothing, because visibility depends only on the descriptor or epoch and snapshots cover only completed epochs.

**I5 SER ordering.** A writer's epoch is fixed before its validation, so every dependency is non-decreasing in epoch.

**I6 Slot publication.** Every reserved slot is written exactly once by its reserver (content, or `VOID`), and a slot is linked into a chain at most once.

*Memory-order requirements.* Each of the following was shown necessary either by a failure or by the memory model. (1) The grower's stores  $B[new] \rightarrow \text{vcursor} \rightarrow \text{vfill}$  are releases, and the reservation `fetch_add` is `acq_rel`. (2) The delta contents reach the chain head through an `acq_rel` CAS, whose release sequence covers later readers. (3) Relaxed loads of delta contents suffice for scanners, because a scanner uses a delta only after establishing its creator's visibility through `desc` or `frontier` (acquire). (4) Commit publication is: fence; relaxed descriptor store; `__syncwarp`; leader fence; RMW on done; acquire load by the advancer; release store of the frontier.

*Progress.* Writers are lock-free per operation, since a CAS failure implies another install succeeded. Aborts are no-wait (first-updater-wins). Livelock under identical-edge contention is possible in principle and is mitigated by randomized exponential backoff. Commit registration is wait-free. The advancer waits only for registered committers, so in SER mode the frontier's staleness is bounded by the longest validation.

#### 5.2 History checker

A final-state comparison ("the logged edge set equals the device's edge set") cannot detect a lost update between two deletions, a torn snapshot, or a dirty read that is later overwritten. Our audit of an earlier GPU prototype found all three, and each passed that check. GTX-GPU is instead validated by a host-side checker over the full history. Let  $S(e)$  be the graph after applying, in *cts* order, the net writes of all committed transactions with  $cts \leq e$ .

- C1 Read rule.** Every read, no-op write and scan result equals  $S(rts)$  overlaid with  $T$ 's own earlier writes; effective writes agree with the same state.
- C2 Observation identity.** A point read names exactly the delta the rule predicts, never an aborted or uncommitted one.
- C3 Write-write.** No committed writer of key  $k$  has  $cts \in (rts(T), cts(T))$  for another writer  $T$  of  $k$ , and no two writers of  $k$  share a commit timestamp.
- C4 Final state.** The device's final edge set equals  $S(\infty)$ .
- C5 Serialization graph (SER).** The direct serialization graph (ww, wr, rw edges; per-key version order = *cts* order; a scan reads every key of its vertex) is acyclic.

*Test suite.* Six adversarial workloads (mixed uniform on 256 vertices, 50% hub mixed, 8 identical hot edges, delete/re-insert churn on 64 vertices with  $K=8$ , 30% hub with  $K=32$ , Zipf with 50% readers) run in eight configurations spanning SI/SER, cooperative reservation on/off, destination/chain conflict granularity, and

**Table 4: Mutation tests: violations reported by each check.**

| Mutation        | Final-state check | History checker                                                                     |
|-----------------|-------------------|-------------------------------------------------------------------------------------|
| no ww check     | passes (0/0)      | 4,386 C3; 10,984 same-epoch writers; 11,507 extra final edges; 6,557 DSG back edges |
| ignore snapshot | passes (0/0)      | 9,658 wrong reads; 5,382 wrong scans; 301,431 DSG back edges                        |
| dirty reads     | 1,133/340         | 11,411 wrong reads; 6,972 wrong scans; 271,175 DSG back edges                       |

per-lane/cooperative scans. All 48 histories pass C1–C5. A *write-skew probe* must show the anomaly under SI and must not show it under SER: SI produced 16,000 serialization-graph back edges, SER produced 0, and both pass their respective checks. A tiny-arena test forces resource exhaustion; the committed prefix remains consistent at every arena size. Every benchmark configuration in §6 also passes the checker in a reduced-size run, and so do all baselines in SER mode.

*Mutation testing.* To show the checker is sensitive, three broken protocols are compiled in (Table 4). The checker rejects all three. The prototype-style final-state check accepts two of them.

### 5.3 Defects found during development

Table 5 lists the protocol defects found by the checker, by stress tests or by memory-model reasoning. All were found and fixed *before* any benchmark in §6 was recorded. D9 teaches a general lesson: a diagnostic mechanism (a watchdog) must never trade safety for liveness.

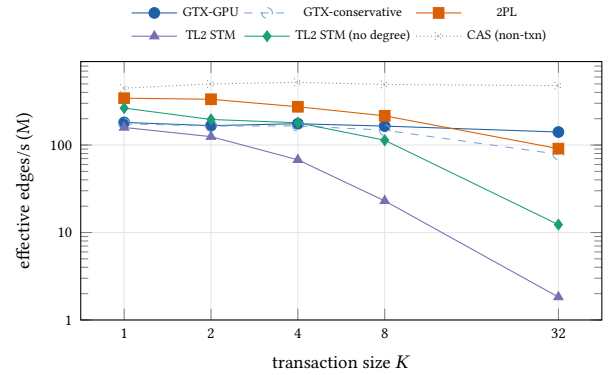
## 6 Evaluation

### 6.1 Setup

*Platform.* NVIDIA RTX 4060 Laptop GPU (Ada, sm\_89, 24 SMs, 8 GB, WDDM), CUDA 12.8, Windows 11, compiled with `nvcc -O3 -arch=sm_89`.

*Systems.* All transactional baselines share one storage layout: per-vertex open-addressing hash adjacency plus a per-vertex degree word, pre-sized from the workload so that, unlike GTX-GPU, they never pay for growth.

- **2PL:** strict two-phase locking with per-vertex reader-writer spinlocks acquired in sorted vertex order (deadlock-free).
- **TL2 STM:** word-based TL2-style STM [8] (global version clock, versioned lock records, lazy write buffering, commit-time sorted locking, read-set validation; read-only transactions without read sets).
- **TL2 STM, no degree:** the same without the degree word. It avoids the hub hot spot but cannot answer degree queries transactionally.
- **Non-transactional CAS:** SlabGraph/fainGraph-style lock-free updates. It provides no atomicity or snapshots and serves as a lower bound on synchronization cost.
- **GTX-conservative:** GTX-GPU with per-lane reservation and GTX’s chain-granularity conflicts, i.e. the closest port of GTX.



**Figure 2: Uniform inserts: effective edge insertions per second vs. transaction size ( $|V|=2^{20}$ , 4M initial edges, 1M operations).**

- **GPMA+** [13]: the authors’ implementation, both a Windows port and the pristine upstream build (within 2% of each other).

*Workloads.* A deterministic generator (splitmix64, seed 1) produces one stream that every system consumes.  $|V| = 2^{20}$ , the initial graph has 4M edges, and each run has about 1M operations ( $2^{20}/K$  transactions of  $K$  operations, or  $2^{18}$  transactions with  $K=4$ ). Sources are uniform, hub (a fraction of operations on one vertex), Zipf( $s=1$ ) or R-MAT. Mixes include inserts, deletes, churn (50/50 insert/delete), reads, identical-edge contention, and 50% or 90% read-only transactions with 10% scans.

*Methodology.* We time the single transactional kernel with CUDA events; initial loading and checking are excluded. Each (scenario, system) runs in its own process with one warm-up and five measured repetitions, and a 90 s timeout is recorded as DNF. We report committed transactions/s and *effective* edge changes/s (never attempted operations). Speedup is the ratio of median committed throughput, with a 95% bootstrap confidence interval over repetitions. Latency is measured per transaction with `%globaltimer` from the first attempt to commit, including retries.

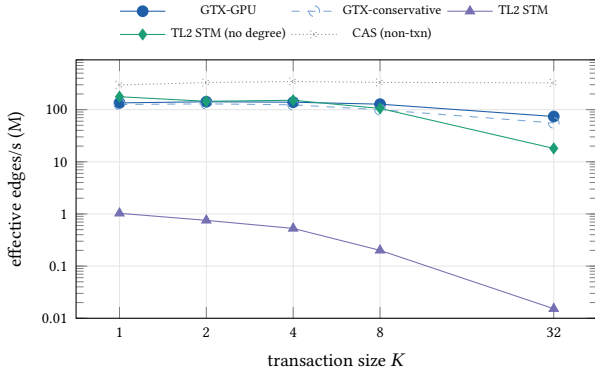
### 6.2 Update throughput and transaction size

Figures 2 and 3 vary the transaction size  $K$  for insert-only workloads. On *uniform* sources (Figure 2), conflicts are rare and uncontended vertex locks are cheap, so 2PL is 1.3–2.0× faster than GTX-GPU up to  $K=8$ . That gap is the fixed cost of versioning (§6.8). As  $K$  grows, lock and read-set footprints grow with it. At  $K=32$ , GTX-GPU sustains 141 M effective edges/s while 2PL drops to 91 M (1.55×), TL2 to 1.8 M (77×), and the conservative GTX port to 79 M, because its chain-granularity rule aborts 45% of attempts.

With a *hub* receiving 50% of operations (Figure 3), 2PL does not finish any  $K$  within 90 s, and TL2 collapses to 0.015–1.0 M edges/s with abort ratios above 99.6% because every insert conflicts on the hub’s degree word. GTX-GPU stays at 74–142 M edges/s. Only the degree-free STM is competitive at small  $K$  (0.76–1.21× of GTX-GPU); at  $K=32$  it aborts 90% of attempts and GTX-GPU is 4.1× faster.

**Table 5: Protocol defects found and fixed before benchmarking.**

| Id | Detected by                   | Root cause                                                                                                                  | Fix                                                                                               | Evidence after fix                          |
|----|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------|
| D5 | 180-rep stress + watchdog     | register by read-increment-reread of E; advancer bumps E $\approx 1/\mu s \Rightarrow$ committer starves (1.5–20 s kernels) | wait-free registration: one <code>fetch_add</code> on (epoch $\ll 32$   count); close by exchange | 180/180 reps in 0.15–1.0 ms                 |
| D6 | checker C5                    | epoch fixed after read validation; rw edge runs backwards in commit order (1–40,535 back edges)                             | Silo order: register epoch before validation                                                      | 0 cycles in all SER histories               |
| D7 | reasoning (C1 risk)           | read-only commit at $cts=rts$ would expose voided slots as committed in the past                                            | read-only commit publishes ABORTED                                                                | suite and check matrix pass                 |
| D8 | forensic scan instrumentation | relaxed reservation RMW: no happens-before to the grower’s generation publication                                           | reservation RMW <code>acq_rel</code>                                                              | memory-order argument; suite pass           |
| D9 | checker C1 + decision codes   | harness watchdog left the advancer’s wait after 20 s and published the frontier early                                       | watchdog only flags; run declared invalid; publishing stops                                       | 4/4 stress iterations and 48 histories pass |

**Figure 3: 50% hub inserts. 2PL did not finish any  $K$  within 90 s (lower-bound speedups of GTX-GPU:  $> 11,000\times$ ).**

### 6.3 Contention, skew and headline comparison

Figure 4 summarizes GTX-GPU’s speedup over each transactional baseline across the scenarios where synchronization is hard, together with the uncontended  $K=1$  case for reference. On power-law churn (insert/delete mixes,  $K=4$ ), 2PL serializes on hub locks: GTX-GPU is  $525\times$  faster on R-MAT, and 2PL does not finish on Zipf. TL2 is  $101\text{--}364\times$  slower. The degree-free STM matches GTX-GPU ( $0.89\text{--}0.96\times$ ) in write-only churn. It loses once readers or large transactions appear.

Under *identical-edge contention* (50% of operations on 64 or 1,024 hot edges), every system is dominated by true conflicts. GTX-GPU is still  $4.6\text{--}12.3\times$  faster than the transactional baselines, because first-updater-wins with no-wait aborts and backoff wastes less work than lock waits or commit-time validation failures.

### 6.4 Concurrent snapshot readers

Readers are where multi-versioning pays off most. In R-MAT with 90% read-only transactions (10% of their operations are adjacency scans), GTX-GPU readers never block writers, never abort and never wait. GTX-GPU commits 17.3 M transactions/s with an overall abort ratio of 0.03%, compared with 90% for TL2. Each baseline was run in two modes, per-lane scans and warp-cooperative read-only scans identical in structure to GTX-GPU’s, and every headline number uses the baseline’s *better* mode. GTX-GPU is  $14.0\times$ ,  $22.7\times$  and  $16.7\times$

**Table 6: GPMA+ vs. GTX-GPU on identical  $K=1$  streams (effective edges/s; GPMA+ Windows port).**

| Stream         | GPMA+ batch |         | GTX-GPU | Speedup              |
|----------------|-------------|---------|---------|----------------------|
|                | 65,536      | 524,288 | (txn)   | (65K / 524K)         |
| uniform insert | 9.03 M      | 12.61 M | 181.4 M | 20.1 / 14.4 $\times$ |
| uniform churn  | 15.26 M     | 17.98 M | 155.7 M | 10.2 / 8.7 $\times$  |
| R-MAT churn    | 15.32 M     | 19.04 M | 143.8 M | 9.4 / 7.6 $\times$   |
| 50% hub insert | 8.43 M      | 10.45 M | 134.7 M | 16.0 / 12.9 $\times$ |

faster than 2PL, TL2 and degree-free TL2. Figure 5 shows read-only tail latency: a p99 of 2.4–2.5 ms for GTX-GPU versus 57–459 ms for the transactional baselines. Under Zipf with 50% readers, 2PL does not finish in either mode, and GTX-GPU is  $8.5\times$  faster than TL2 and  $2.5\times$  faster than degree-free TL2.

Cooperative scanning is required, though. With per-lane scans, GTX-GPU takes 2,577 ms on Zipf readers, versus 438 ms with warp scans, because a scan reads every 32-byte version, including aborted and voided slots.

### 6.5 Serializable transactions

In SER mode with 20% read-only transactions and point reads inside update transactions (checked configuration,  $|V|=2^{16}$ ), GTX-GPU is  $652\times$ ,  $119\times$  and  $20\times$  faster than 2PL, TL2 and degree-free TL2 on Zipf, and  $59\times$ ,  $23\times$  and  $10\times$  faster on a 50% hub. Every system passes C1–C5 on these runs. At full size ( $|V|=2^{20}$ , read-write  $K=4$ ), SER costs GTX-GPU 10–17% throughput relative to SI (e.g. 46.0 M vs 50.9 M transactions/s on uniform). 2PL does not finish on the hub and is  $1,471\times$  slower on Zipf. On uniform read-write, SER GTX-GPU reaches  $0.62\times$  of 2PL and  $2.26\times$  of TL2.

### 6.6 Comparison with GPMA+

GPMA+ applies a sorted batch as one unit and offers no multi-edge transactions or concurrent readers. We fed it the exact  $K=1$  streams used for GTX-GPU (Table 6). GTX-GPU commits every operation as its own transaction and is still  $8.6\text{--}14.6\times$  faster than GPMA+ with 524K-edge batches and  $10\text{--}20\times$  with 65K-edge batches. GPMA+ also loses intra-batch operation order, so on hub and churn streams (duplicates, or insert and delete of one edge within a batch) its final state differs from sequential semantics.

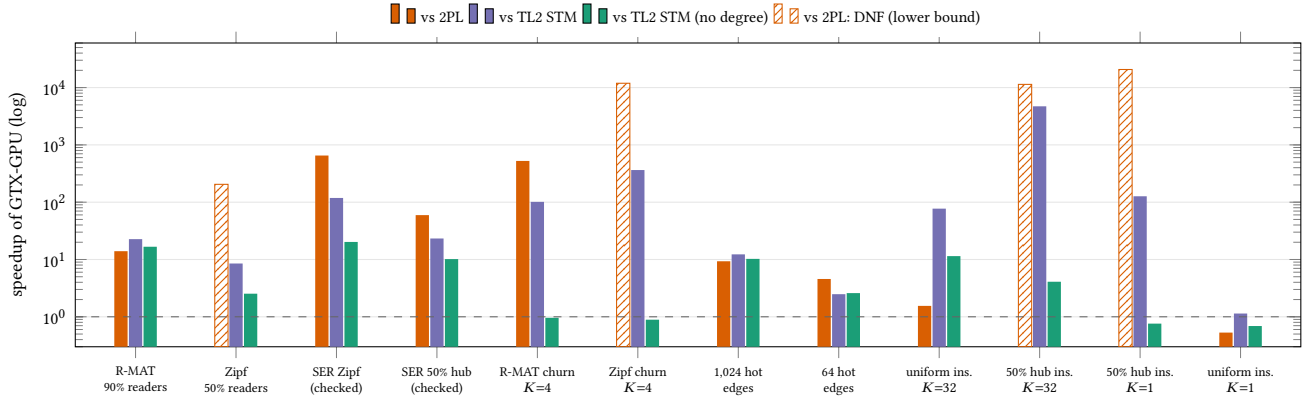


Figure 4: Speedup of GTX-GPU in committed transactions/s over each transactional baseline ( $> 1$ : GTX-GPU faster). Hatched bars: 2PL did not finish within the timeout; the bar is a lower bound that assumes 2PL would have committed everything at the timeout. Reader scenarios use each baseline’s best scan mode. SER scenarios are the checked configurations ( $|V|=2^{16}$ ).

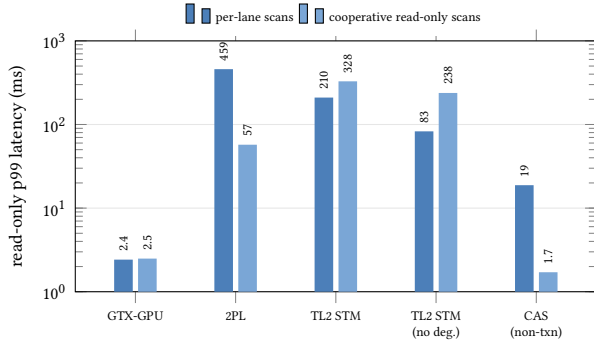


Figure 5: Read-only transaction p99 latency, R-MAT with 90% readers,  $K=4$ . GTX-GPU scans cooperatively in both bars; the modes refer to the baselines.

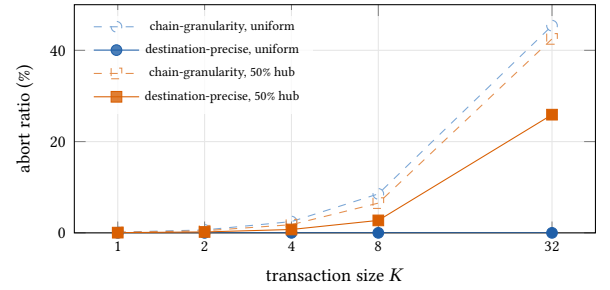


Figure 6: Abort ratio under GTX’s chain-granularity rule (GTX-conservative) and GTX-GPU’s destination-precise validate-then-CAS (insert-only workloads).

## 6.7 Ablation and design hypotheses

Each GPU mechanism was stated with a falsification criterion before measurement (Table 7).

**M1: cooperative reservation** (gtx vs gtx-dst-only, identical except coop). The hypothesis was a  $\geq 10\%$  gain on hub workloads. Measured gains were +8.6% and +11.0% on 50% hub at  $K=1$  and  $K=8$ , +6.0% and +5.8% on 100% hub, and +3.4% on uniform  $K=8$ . Three of the four hub cells are below threshold, so M1 is *falsified as a primary mechanism*. Same-address RMWs on Ada’s L2 are not the hub bottleneck. It is kept as a small win that was never negative.

**M1b: destination-precise conflicts** (gtx vs gtx-coop-only and gtx-cons). This is *supported*. Figure 6 shows the abort ratio: chain-granularity aborts grow with  $K$  to 45% (uniform) and 43% (hub) at  $K=32$ , while destination-precise detection has zero aborts on uniform and 26% on the hub (true conflicts). Throughput improves  $\times 1.14$  at uniform  $K=8$  (isolated effect), and  $\times 1.80$  (uniform) and  $\times 1.33$  (hub) at  $K=32$  over the conservative port.

**M3: epoch commit groups**. *Not falsified*. Commit takes 9.6% (4M-edge initial graph) to 15.5% (empty graph) of the lane-cycle

Table 7: Design hypotheses and outcomes.

| Method                            | Falsified if                                      | Outcome                         |
|-----------------------------------|---------------------------------------------------|---------------------------------|
| M1 cooperative reservation        | $<10\%$ gain on hub, $K \in \{1, 8\}$             | falsified (+3.4–11%)            |
| M1b destination-precise conflicts | $<10\%$ gain at $K \geq 8$ or any checker failure | supported                       |
| M2 multiple reservation regions   | $<10\%$ gain on 100% hub                          | not built (ranked low after M1) |
| M3 epoch commit groups            | commit $>25\%$ of cycles, or C1–C5 failure        | not falsified (9.6–15.5%)       |
| M4 reclamation by generation      | $>20\%$ throughput cost                           | specified, not built            |

profile for uniform  $K=1$  inserts, below the 25% threshold, and all histories pass.

Two micro-optimizations were also falsified: padding hot counters onto separate cache lines (no gain, reverted), and relaxed versus acq\_rel reservation RMW (no measurable difference, so the fence required by D8 is free here).

**Table 8: Scenarios where GTX-GPU is slower (GTX-GPU / baseline throughput).**

| Scenario                                         | Ratio          | Reason                                                                                                                |
|--------------------------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------------|
| uniform ins./churn/del., $K \leq 8$ , vs 2PL     | 0.47–0.76      | versioning cost (32 B delta, chain head, descriptor/epoch); uncontended locks are cheap                               |
| write-only contention vs TL2 without degree word | 0.64–0.98      | hash-slot writes rarely conflict without a degree word; this STM cannot serve degree queries and collapses with scans |
| any scenario vs non-txn CAS                      | 0.10–0.45      | CAS offers no atomicity or snapshots                                                                                  |
| memory                                           | 2.5–20× larger | aborted and voided deltas never reclaimed (e.g. 1.5 GB vs 77 MB, 64 hot edges)                                        |

## 6.8 Where GTX-GPU loses

Table 8 lists every scenario in which GTX-GPU is slower. The uncontended gap to 2PL has no single hotspot. The engine skeleton alone (about 3.3 ms per 1M operations) takes as long as 2PL’s entire run. Closing the gap requires fewer bytes per version and cheaper per-attempt bookkeeping.

## 6.9 Threats to validity

The baselines’ hash tables are pre-sized and never grow, which favours them. Scans were compared in both per-lane and cooperative modes, and the better mode of each baseline is reported. DNF lower bounds assume the baseline would have finished at the time-out. Literature numbers (TITAN V, P100, 2080 Ti, other datasets) are used only as reference points and never as head-to-head claims. All experiments use synthetic generators on one laptop GPU under WDDM. Real SNAP graphs and batch-size sweeps are future work.

## 7 Limitations and Future Work

- (1) **Reclamation and consolidation by generation (M4).** A generation becomes reclaimable when it is sealed for all chains, every delta in it is superseded at an epoch  $\leq \text{minActiveRts}$  (or aborted or voided), and no reader with  $\text{rts} < \text{its}$  retire epoch is active. Protection is tracked per generation rather than per delta. Live latest versions are consolidated into a fresh generation under the same sealing protocol, and backpressure on long scans and arena watermarks bounds memory. This is the main remaining gap: it removes the memory overhead and shortens scans.
- (2) **Closing the uncontended gap:** split hot and cold delta fields, and make per-attempt bookkeeping cheaper.
- (3) **Broader comparisons:** real graphs, batch-size sweeps from  $2^{10}$  to  $2^{22}$ , and local builds of LPMA, Hornet, faimGraph and SlabGraph.
- (4) **Out of scope:** vertex insertion and deletion, descriptor reuse (needs generation-tagged ids), multiple reservation regions (M2), and durability.

## 8 Conclusion

GTX-GPU shows that GTX’s architecture (versioned edge deltas, destination-indexed delta chains, lazily resolved descriptors and epoch group commit) transfers to the GPU, provided four parts are redesigned for SIMT execution: conflict detection must validate and

install on the same observed head, blocks must grow by generations instead of copying, commit registration must be a single wait-free RMW, and serializable validation must follow the epoch. The result offers multi-edge transactions and non-blocking snapshot readers, which no existing GPU dynamic-graph structure provides. It outperforms lock-based and word-STM synchronization by one to four orders of magnitude where synchronization is hard, and it pays a bounded, measured cost where it is easy. A history checker that validates every operation was essential. It found five protocol defects that a final-state comparison would have missed, and we consider it a necessary part of any claim about GPU transaction protocols.

## References

- [1] Atul Adya. 1999. *Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions*. Ph.D. Dissertation. Massachusetts Institute of Technology.
- [2] Saman Ashkiani, Martin Farach-Colton, and John D. Owens. 2018. A Dynamic Hash Table for the GPU. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*.
- [3] Muhammad A. Awad, Saman Ashkiani, Serban D. Porumbescu, and John D. Owens. 2020. Dynamic Graphs on the GPU. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*.
- [4] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. 1995. A Critique of ANSI SQL Isolation Levels. In *ACM SIGMOD International Conference on Management of Data*.
- [5] Federico Busato, Oded Green, Nicola Bombieri, and David A. Bader. 2018. Hornet: An Efficient Data Structure for Dynamic Sparse Graphs and Matrices on GPUs. In *IEEE High Performance Extreme Computing Conference (HPEC)*.
- [6] Michael J. Cahill, Uwe Röhm, and Alan D. Fekete. 2008. Serializable Isolation for Snapshot Databases. In *ACM SIGMOD International Conference on Management of Data*.
- [7] Kevin Jude Concessao, Unnikrishnan Cheramangalath, M. J. Ricky Dev, and Rupesh Nasre. 2023. Meerkat: A Framework for Dynamic Graph Algorithms on GPUs. *arXiv preprint arXiv:2305.17813* (2023).
- [8] Dave Dice, Ori Shalev, and Nir Shavit. 2006. Transactional Locking II. In *International Symposium on Distributed Computing (DISC)*.
- [9] Per Fuchs, Domagoj Margan, and Jana Giceva. 2022. Sortedlton: A Universal, Transactional Graph Data Structure. *Proceedings of the VLDB Endowment* 15, 6 (2022).
- [10] Dean De Leo and Peter Boncz. 2021. Teseo and the Analysis of Structural Dynamic Graphs. *Proceedings of the VLDB Endowment* 14, 6 (2021).
- [11] Yinnian Lin, Lei Zou, and Xunbin Su. 2025. Towards Sufficient GPU-accelerated Dynamic Graph Management: Survey and Experiment. *Proceedings of the VLDB Endowment* 18 (2025), 599.
- [12] Diogo Nunes, Daniel Castro, and Paolo Romano. 2022. CSMV: A Highly Scalable Multi-Versioned Software Transactional Memory for GPUs. In *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 526–536.
- [13] Mo Sha, Yuchen Li, Bingsheng He, and Kian-Lee Tan. 2017. Accelerating Dynamic Graph Analytics on GPUs. *Proceedings of the VLDB Endowment* 11, 1 (2017), 107–120.
- [14] Mo Sha, Yuchen Li, Bingsheng He, and Kian-Lee Tan. 2018. Technical Report: Accelerating Dynamic Graph Analytics on GPUs. *arXiv preprint arXiv:1709.05061v2* (2018).
- [15] Qi Shen, Craig Sharp, William Blewitt, Gary Ushaw, and Graham Morgan. 2015. PR-STM: Priority Rule Based Software Transactions for the GPU. In *Euro-Par 2015: Parallel Processing*.
- [16] Stephen Tu, Wenting Zheng, Eddie Kohler, Barbara Liskov, and Samuel Madden. 2013. Speedy Transactions in Multicore In-Memory Databases. In *ACM Symposium on Operating Systems Principles (SOSP)*.
- [17] Martin Winter, Daniel Mlakar, Rhaleb Zayer, Hans-Peter Seidel, and Markus Steinberger. 2018. faimGraph: High Performance Management of Fully-Dynamic Graphs Under Tight Memory Constraints on the GPU. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 754–766.
- [18] Yunlong Xu, Rui Wang, Nilanjan Goswami, Tao Li, Lan Gao, and Depei Qian. 2014. Software Transactional Memory for GPU Architectures. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*.
- [19] Libin Zhou, Lu Xing, Yeasir Rayhan, and Walid G. Aref. 2024. GTX: A Write-Optimized Latch-free Graph Data System with Transactional Support – Extended Version. *arXiv preprint arXiv:2405.01418v2* (2024). SIGMOD 2025.
- [20] Xiaowei Zhu, Guanyu Feng, Marco Serafini, Xiaosong Ma, Jiping Yu, Lei Xie, Ashraf Aboulmaga, and Wenguang Chen. 2020. LiveGraph: A Transactional Graph Storage System with Purely Sequential Adjacency List Scans. *Proceedings of the VLDB Endowment* 13, 7 (2020).

- [21] Lei Zou, Fan Zhang, Yinnian Lin, and Yanpeng Yu. 2023. An Efficient Data Structure for Dynamic Graph on GPUs. *IEEE Transactions on Knowledge and Data Engineering* (2023). PKU preprint.

## A Reproducing the Results

All code, raw data and scripts are in the gtx-gpu-research repository.

- Correctness gate: `build.bat test_gtx && bin\test_gtx.exe full`.
- Benchmarks: `build.bat bench;python scripts/run_matrix.py main|ser|ablation|fair|fairro|gpmamatch|check;python scripts/summarize.py (medians, bootstrap CIs, results/summary.md, results/speedups.csv)`.
- GPMA+: `baselines/gpma/build_gpma.bat;python scripts/run_gpma`
- Source map: `src/gtxg.cuh` (engine), `src/baselines.cuh` (2PL, TL2, CAS), `src/checker.hpp` (C1–C5), `src/workload.hpp` (generator), `src/bench.cu`, `src/test_gtx.cu`.